



Continual Learning on Recommendation Systems

2026-01-22
HoonUi Lee

Contents

❖ Continual Learning

- ❑ What is Continual Learning?
- ❑ Continual Learning on RS
- ❑ Experience-Replay
- ❑ Regularization

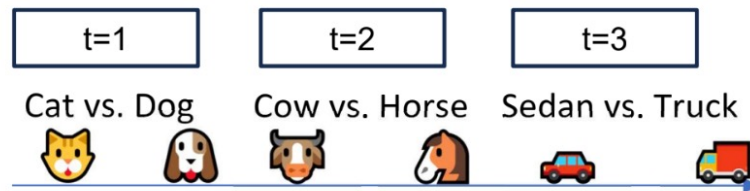
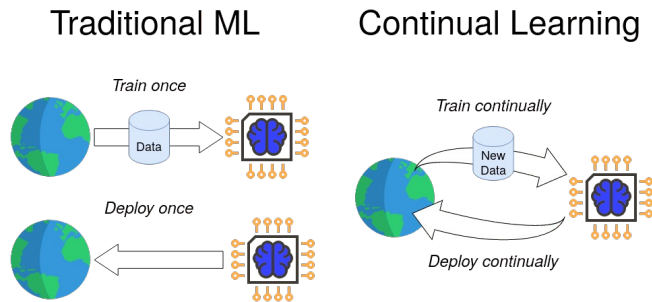
❖ INFER

❖ Graph-SANE

Continual Learning

❖ New tasks or data arrive sequentially, model incrementally learns from new data

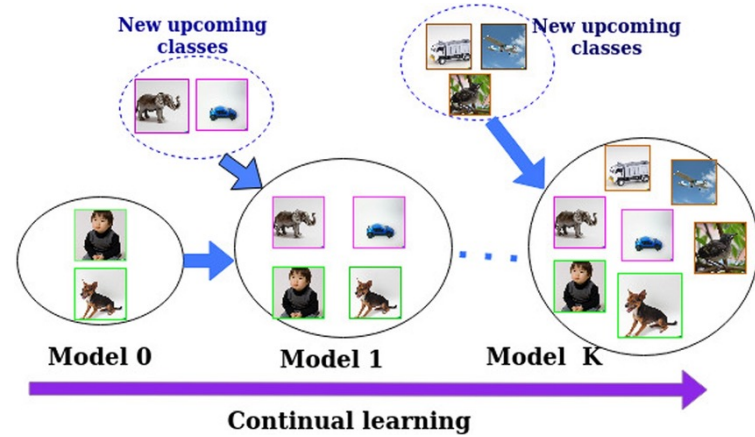
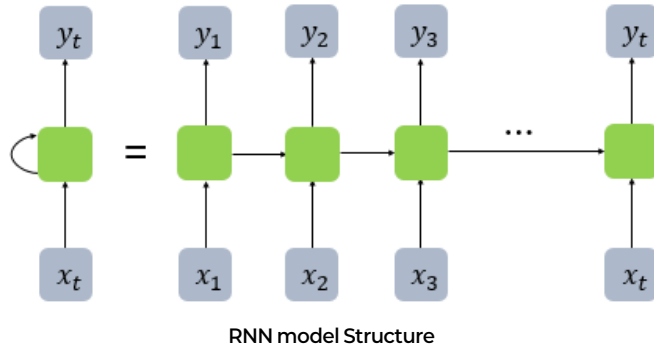
- ❑ Forgetting: as the model learns new tasks, it forgets old knowledge
- ❑ Goal: be good on both past and new tasks



❖ Core challenge

- ❑ Balance between **stability** (preserving past knowledge) and **plasticity** (adapting to new knowledge)

Continual vs Sequential



❖ Sequential Learning

- Inputs (data) are arranged in a time-ordered / sequential manner (Model structure)

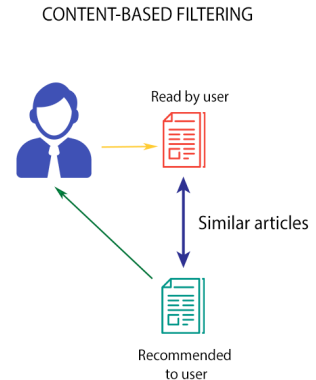
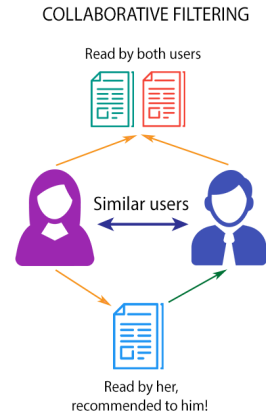
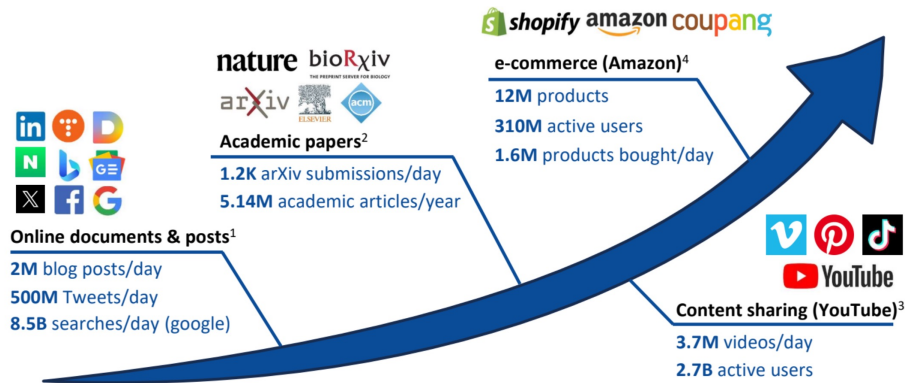
❖ Continual Learning

- The model keeps learning with new upcoming data (updating) over time, while preserving and managing previously learned knowledge as much as possible (Learning setting)

Recommendation System

❖ The Era of Information Explosion

- Recommend items that fit each user to support effective decision-making



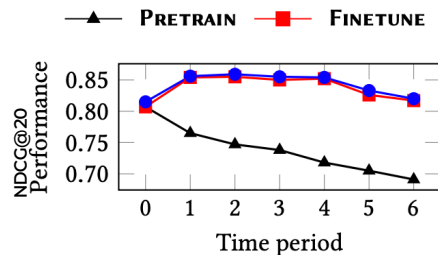
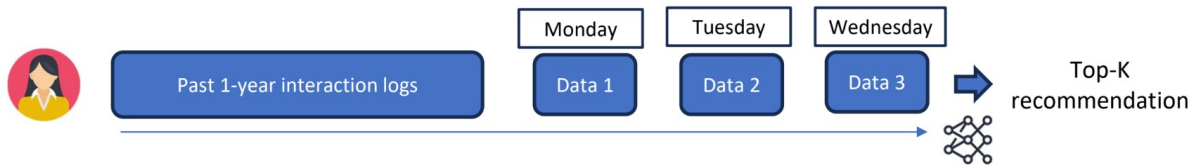
❖ Make recommendations by leveraging diverse information source

- User profile, item profile, collaborative info., user-item interaction history, ...

Continual Learning on RS

❖ Suppose we trained recommendation models on Sunday using the past 1 year of data

- ❑ It works well for Monday, but as days pass, the model becomes stale
 - User preferences continuously evolve; new users and items

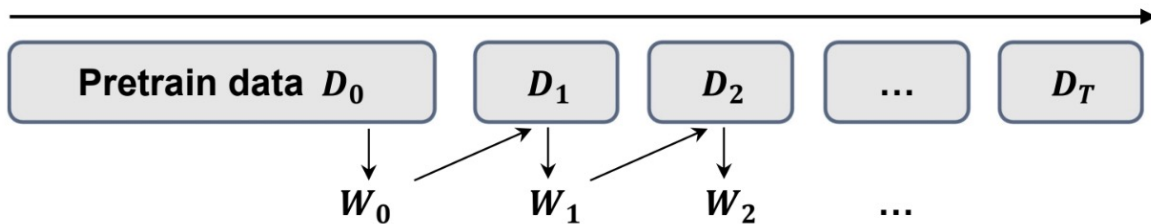


❖ How to update the model with newly coming data?

- ❑ Periodic retraining with all data (historical+new) -> highly **time-inefficient**
- ❑ Incremental fine-tuning only with new data -> highly **efficient and adaptable!**

➔ About Plasticity

Notation for Continual Recommendation Systems



❖ **Data block at t :** $D_t = \{U_t, I_t, E_t\}, \forall t \in \{1, \dots, T\}$

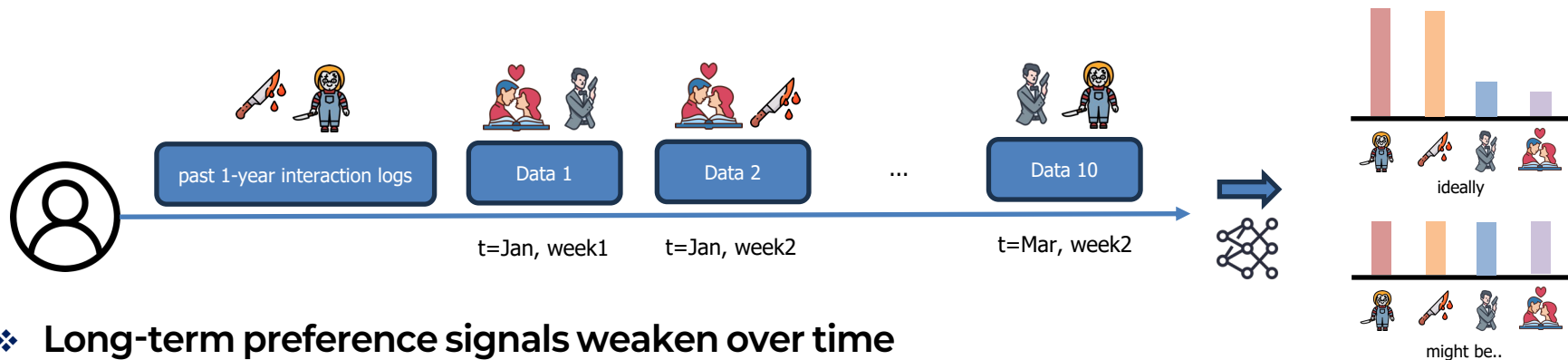
□ U_t : user set, I_t : item set, E_t : user-item interaction set

❖ **Model parameter W_t**

□ Each user and item has its own embedding (representation)

□ Or the parameters of the user and item embedding encoders (e.g., the two-tower encoders)

Issues with Pure Fine-tuning

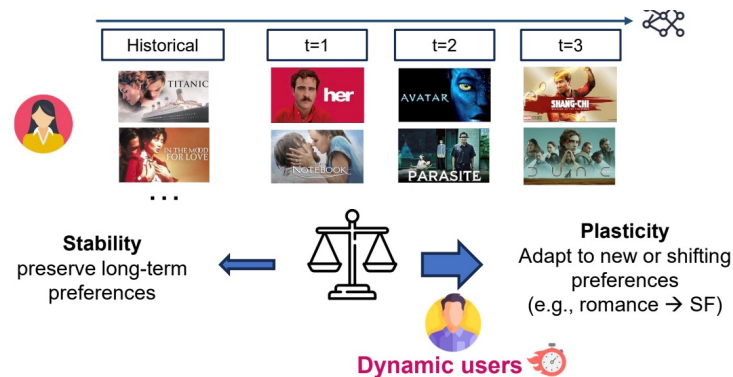
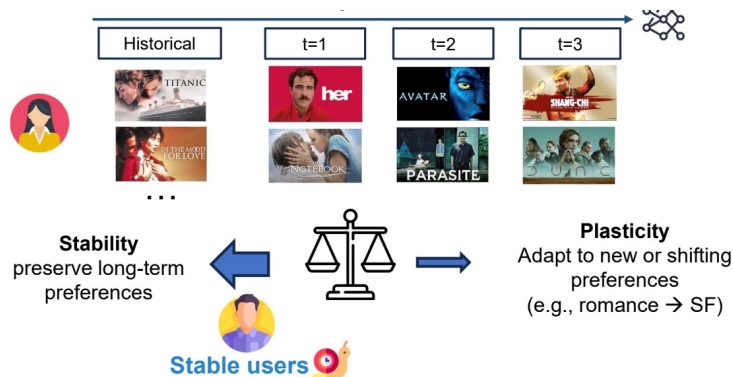


❖ Long-term preference signals weaken over time

- ❑ Pure fine-tuning optimizes only for recent data
- ❑ Recommendation needs both short-term interest and **recurring long-term preferences**
 - In an $A \rightarrow B \rightarrow A$ shift, recovery can be slow and unstable
- ❑ We need mechanisms to preserve long-term preferences

➔ About Stability

Stability vs Plasticity



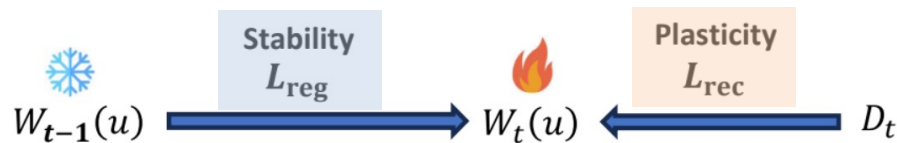
❖ Users differ in how much their preferences shift

- ❑ Do a user's preferences stay stable or change dynamically?
- ❑ When we prioritize stability or plasticity?

Regularization-based Method

❖ Constrain current parameters to stay close to past ones

- Regularize parameters learned from new data to stay close to those from previous stages, preserving past knowledge



$$L := L_{\text{rec}}(D_t; W_t) + \lambda \sum_{u \in U_{:t}} L_{\text{reg}}(W_t(u), W_{t-1}(u)),$$

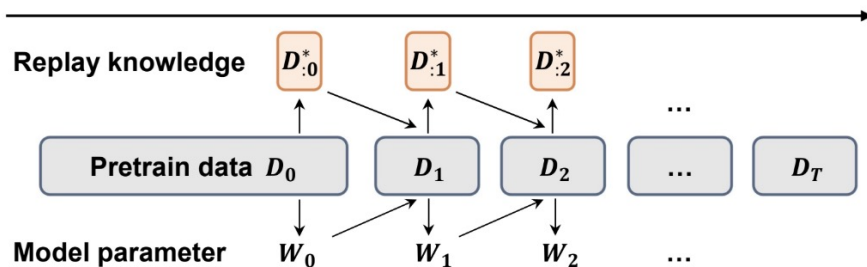
Init $W_t \leftarrow W_{t-1}$

❖ Most prior work focuses on preserving the structure of the user-item graph

- user-item, user-user, item-item, ...

Experience-Replay-based Method

- ❖ Use a subset of past data that is representative or useful
 - ❑ Can't store the entire history
 - Inefficient, and even stored data can be costly to use
 - ❑ Replay size γ is typically small (e.g., $\gamma = 1$) compared to full history



$$\mathcal{L} := \mathcal{L}_{\text{rec}}(\mathcal{D}_t \cup \mathcal{D}_{:t-1}^*; \mathcal{W}_t), |\mathcal{D}_{:t-1}^*| = \gamma |\mathcal{D}_t|,$$

- ❖ Main problem: Define **what “useful” means** and selecting samples

Upcoming Topics

❖ How to select replay samples for stability

- ❑ Data-level properties, Model-level properties
- ❑ What makes a sample good for long-term preference?

❖ Plasticity strategies for dynamic users

- ❑ How can we **detect and capture** preference shifts effectively?
- ❑ Once detected, how do we **incorporate** them into training?



Influential Exemplar Replay for Incremental Learning in Recommender System

Xinni Zhang , Yankai Chen , Chenhao Ma , Yixiang Fang , Irwin King

The Chinese University of Hong Kong

The Chinese University of Hong Kong, Shenzhen

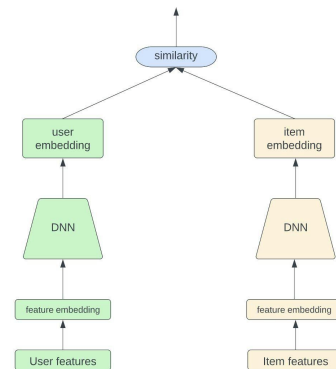
INFER (AAAI' 24)

❖ Effective model-agnostic incremental learning framework

- ❑ Plug-and-play with diverse recommender backbones
- ❑ Many recent incremental recommender methods are graph-based
 - Limited backbone compatibility

❖ Retrieve influential exemplars from historical interactions

- ❑ Prior replay methods: data-level heuristics
- ❑ INFER: model-level influence
 - Pick samples that most help retain previous knowledge under current model state



Two-tower RS model

Data-level Property-based Selection

❖ Uniform selection

- Randomly samples and replays a subset of old data

$$\{(u, i) \sim U(\mathcal{H})\}$$

❖ Inverse degree

- Each interaction's selection probability proportional to the inverse degree of its user
 - d_u : the degree of user u
 - T : temperature

$$P\{(u, i)\} = \frac{1/d_u^T}{\sum_{(\hat{u}, \hat{i}) \in \mathcal{H}} 1/d_{\hat{u}}^T}$$

Data-level Property-based Selection (Cont.)

❖ Frequency & feature distribution

- Step 1: determine the number of replay samples per item (frequency)
 - Prioritize recently frequent & popular items

$$N \cdot \frac{|\{(x, y = i) \in \overset{\text{Current block}}{D_t} \cup \overset{\text{Previous replay buffer}}{E_{t-1}}\}|}{|D_t \cup E_{t-1}|}$$

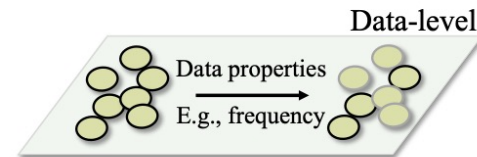
- Step 2: select representative samples (feature distribution)
 - Ensures that samples best represent the item's overall feature distribution

$$x^k \leftarrow \arg \min_{x \in \mathcal{P}_y} \left\| \overset{\text{Avg. feature of items}}{\mu} - \frac{1}{k} \left(\phi(x) + \sum_{j=1}^{k-1} \phi(x^j) \right) \right\|$$

How to select influential samples?

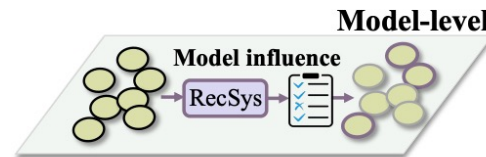
❖ Data-level properties

- ❑ Select samples based on data statistics
 - Frequencies or diversities



❖ Model-level properties

- ❑ Select samples that help the model “not forget”
 - Goal: maintain performance on the reservoir (representative past data)
- ❑ Influence-based selection
 - Under the current model θ_t , score each candidate by **how the Reservoir loss changes** if we **slightly upweight** that sample
 - Keep samples that **reduce** the Reservoir loss (the **most negative** influence)
 - Influence: small change in data occurrence perturbs recommendation performance



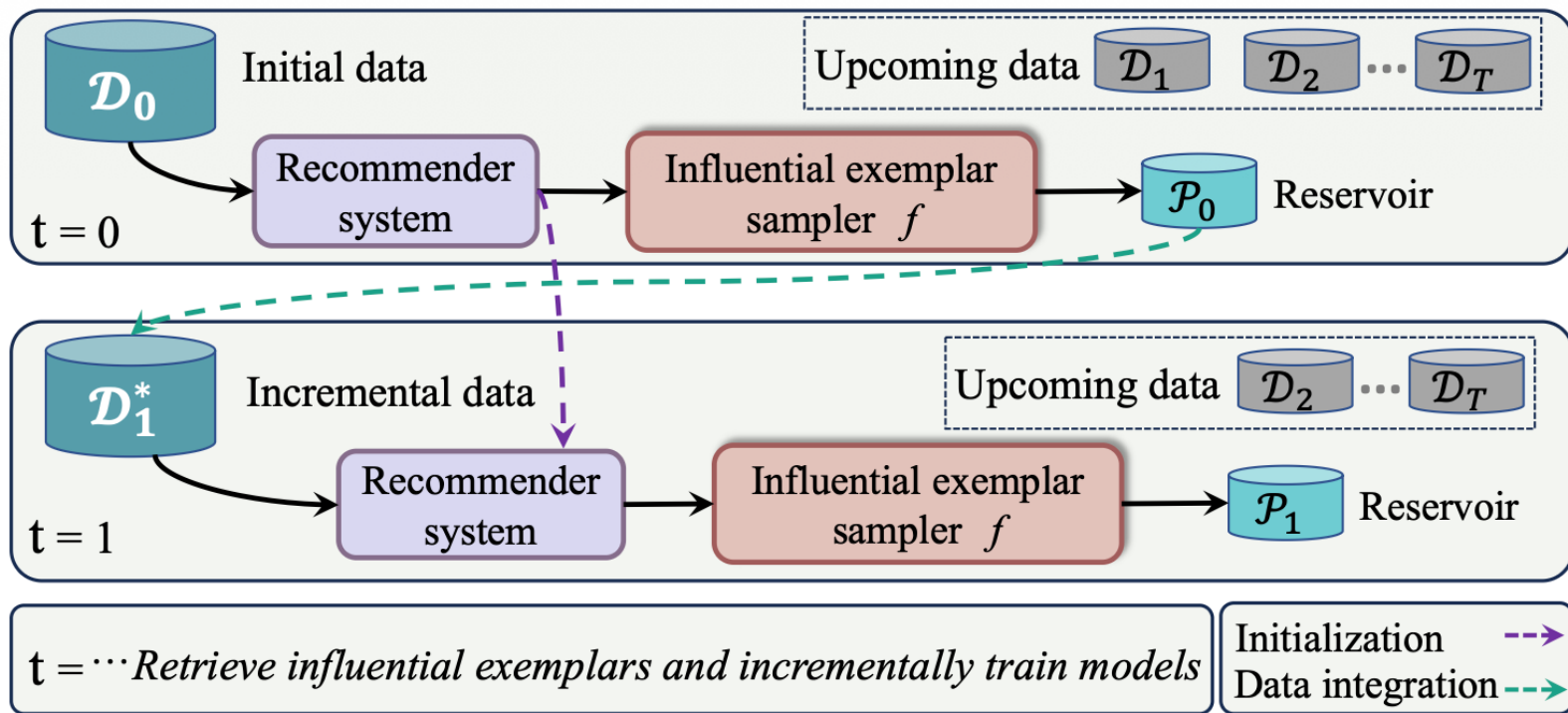
previously
sampled data
(Reservoir)

$$D_t^* := P_{t-1} \cup D_t$$

$$P_t := f(\hat{\theta}_t | K, D_t^*).$$

current model

INFER (INfluential Exemplar Replay)



Vanilla Solution : Influence-based Replay

$$\hat{\theta}_t := \arg \min_{\theta_t \in \Theta} \sum_{z_i \in \mathcal{D}_t^*} \mathcal{L}(z_i, \theta_t), \quad (2)$$



$$\hat{\theta}_{t,\epsilon,z_m} := \arg \min_{\theta_t \in \Theta} (\epsilon \mathcal{L}(z_m, \theta_t) + \sum_{z_i \in \mathcal{D}_t^*} \mathcal{L}(z_i, \theta_t)). \quad (3)$$

$\mathcal{D}_t^* := P_{t-1} \cup D_t$

❖ Influence function measures effect of one observation on an estimator

- Observe the impact of upweighting z_m by some small perturbation ϵ
- $\hat{\theta}_{t,\epsilon,z_m}$ denote the optimal parameters under this perturbation

Vanilla Solution : Influence-based Replay (Cont.)

$$\mathcal{I}_{\text{params}}(z_m) := \left. \frac{d\hat{\theta}_{t,\epsilon,z_m}}{d\epsilon} \right|_{\epsilon=0} = -H_{\hat{\theta}_t}^{-1} \nabla_{\theta_t} \mathcal{L}(z_m, \hat{\theta}_t), \quad (4)$$

❖ Estimating the influence of z_m on the parameters

- Approximated using the inverse Hessian and the sample's gradient

$$\mathcal{I}_{\text{rec_loss}}(z_m) := \sum_{\substack{z_i \in \mathcal{D}_{0:t} \\ D_t^* := P_{t-1} \cup D_t}} \left. \frac{d\mathcal{L}(z_i, \hat{\theta}_{t,\epsilon,z_m})}{d\epsilon} \right|_{\epsilon=0}. \quad (5)$$

❖ In recommendation, we care about preserving previously learned knowledge

- If we slightly upweight z_m , how much does the total loss on past (representative) samples change?
 - Loss **decreases** → z_m **supports** retaining/strengthening past knowledge
 - Loss **increases** → z_m **conflicts** with past knowledge and may cause forgetting

Vanilla Solution : Influence-based Replay (Cont.)

$$\begin{aligned}
 \mathcal{I}_{\text{rec_loss}}(z_m) &\approx \sum_{z_i \in \mathcal{P}_{t-1} \cup \mathcal{D}_t} \left. \frac{d\mathcal{L}(z_i, \hat{\theta}_{t,\epsilon,z_m})}{d\epsilon} \right|_{\epsilon=0} \\
 &= \sum_{z_i \in \mathcal{D}_t^*} \nabla_{\theta_t} \mathcal{L}(z_i, \hat{\theta}_t)^\top \cdot \mathcal{I}_{\text{params}}(z_m).
 \end{aligned} \tag{6}$$

❖ Influence on predictions: “how z_m affects the losses of other samples”

□ Final form via the chain rule

- Under the current model $\hat{\theta}_t$, it becomes an **inner product**:
(direction that increases z_i 's loss fastest) · (parameter update induced by z_m)
- Two directions are **opposite** → inner product < 0 → upweighting z_m tends to **decrease** the loss of z_i .

□ Select the interactions with the K most negative influence values

INFER_ONCE: One-step Influence Estimation

❖ Limitations of the vanilla implementation

- ❑ Computing the inverse Hessian is extremely expensive

- Quadratic in the number of parameters

$$\begin{aligned} \mathcal{I}_{\text{rec.loss}}(z_m) &\approx \sum_{z_i \in \mathcal{P}_{t-1} \cup \mathcal{D}_t} \left. \frac{d\mathcal{L}(z_i, \hat{\theta}_{t,\epsilon,z_m})}{d\epsilon} \right|_{\epsilon=0} \quad (6) \\ &= \sum_{z_i \in \mathcal{D}_t^*} \nabla_{\theta_t} \mathcal{L}(z_i, \hat{\theta}_t)^\top \cdot \mathcal{I}_{\text{params}}(z_m). \end{aligned}$$

$\mathcal{I}_{\text{params}}(z_m) := \left. \frac{d\hat{\theta}_{t,\epsilon,z_m}}{d\epsilon} \right|_{\epsilon=0} = -H_{\hat{\theta}_t}^{-1} \nabla_{\theta_t} \mathcal{L}(z_m, \hat{\theta}_t)$

❖ Fast Influence Estimation

$$\hat{\theta}_{t,\epsilon,z_m} := \theta_t - \lambda \nabla_{\theta_t} (\epsilon \mathcal{L}(z_m, \theta_t) + \sum_{z_i \in \mathcal{D}_t^*} \mathcal{L}(z_i, \theta_t)). \quad (8)$$

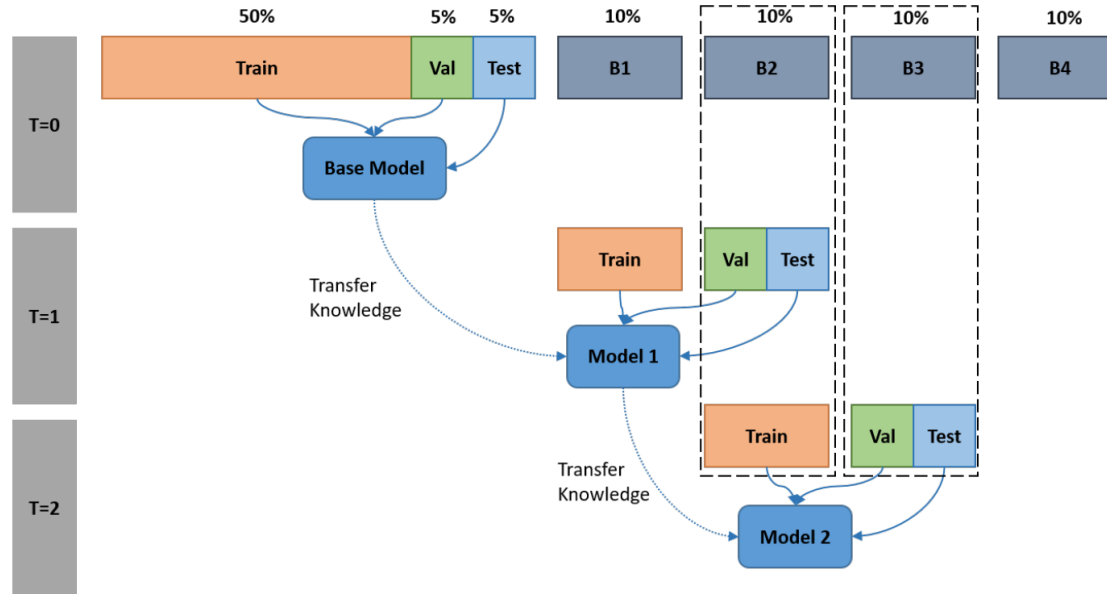
$$\begin{aligned} \mathcal{I}_{\text{rec.loss}}^*(z_m) &:= \sum_{z_i \in \mathcal{D}_t^*} \left. \frac{d\mathcal{L}(z_i, \hat{\theta}_{t,\epsilon,z_m})}{d\epsilon} \right|_{\epsilon=0} \\ &= \sum_{z_i \in \mathcal{D}_t^*} \nabla_{\theta_t} \mathcal{L}(z_i, \hat{\theta}_t)^\top \cdot \left. \frac{d\hat{\theta}_{t,\epsilon,z_m}}{d\epsilon} \right|_{\epsilon=0} \quad (9) \\ &= -\lambda \sum_{z_i \in \mathcal{D}_t^*} \nabla_{\theta_t} \mathcal{L}(z_i, \hat{\theta}_t)^\top \cdot \nabla_{\theta_t} \mathcal{L}(z_m, \theta_t). \end{aligned}$$

- ❑ Approximate the parameter change induced by z_m with a **one-step update** at the current θ_t .

- Hessian -> Gradient

Experiment Setup

❖ Evaluation Benchmarks



Evaluation Metrics

❖ CTR prediction

- Predict the probability of a click (user–item interaction).
- AUC
 - The probability that a randomly chosen **clicked (positive)** sample is scored higher than a randomly chosen **non-clicked (negative)** sample

❖ Top-N recommendation

- Evaluate how many relevant items appear in the top- N list
- Recall@N
 - If the user has a relevant items and b of them are in the top- N , then $\text{Recall@N} = b/a$

❖ Report the average test performance across incremental update steps

Competing Algorithms

❖ Conventional Implementations

❑ Fine-tune

- Fine-tune the pretrained model using only the new incremental data
 - Lower bound for incremental performance

❑ FD-Retrain

- Retrain from scratch using all accumulated data at each update.
 - Upper bound for incremental performance

❖ Incremental Recommender Models

❑ GraphSAIL, SGCT, LWC-KD

- Graph-based SOTA incremental recommenders
- Use techniques such as **knowledge distillation** to preserve key knowledge from past data

Competing Algorithms (Cont.)

❖ Experience Replay Methods

□ Uniform

- Randomly sample a subset of past data **uniformly** and replay it during training

□ ER-MIR

- **Predict parameter updates** and select the most “interfering/important” samples for replay

□ InvDeg

- Sample using **graph statistics** (e.g., inverse degree)
- Can be **extended beyond graph settings**

Overall Performance

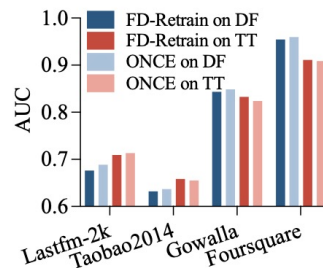
Task	Backbone Method	Lastfm-2k		Taobao2014		Gowalla		Foursquare	
		DeepFM	TwoTower	DeepFM	TwoTower	DeepFM	TwoTower	DeepFM	TwoTower
CTR	Fine-Tune	0.6858	0.7054	0.6356	0.6472	0.8403	0.7836	0.9569	0.9000
	Uniform	0.6814	0.7091	0.6367	0.6495	0.8450	0.8093	0.9585	0.9057
	ER-MIR	0.6823	0.7097	0.6330	0.6510	0.8441	0.8174	0.9587	0.9054
	InvDeg	0.6766	0.6951	0.6351	0.6502	0.8448	0.8035	0.9598	0.8993
	INFER _{Vanilla}	0.6863	0.7099	0.6338	0.6528	0.8432	0.8052	0.9547	0.9062
	INFER _{ONCE}	0.6884	0.7134	0.6370	0.6551	0.8486	0.8237	0.9595	0.9087
Top- <i>N</i>		BPR-MF	LightGCN	BPR-MF	LightGCN	BPR-MF	LightGCN	BPR-MF	LightGCN
	Fine-Tune	0.0450	0.0479	0.0066	0.0073	0.0573	0.0752	0.0842	0.1074
	Uniform	0.0469	0.0660	0.0063	0.0081	0.0588	0.0838	0.0913	0.1189
	ER-MIR	0.0475	0.0668	0.0069	0.0076	0.0587	0.0823	0.0908	0.1089
	InvDeg	0.0479	0.0655	0.0067	0.0079	0.0602	0.0845	0.0922	0.1200
	GraphSAIL	-	0.0553	-	0.0078	-	0.0725	-	0.1042
	SGCT	-	0.0620	-	0.0080	-	0.0783	-	0.1157
	LWC-KD	-	0.0674	-	0.0081	-	0.0792	-	0.1194
	INFER _{Vanilla}	0.0513	0.0695	0.0074	0.0083	0.0591	0.0847	0.0913	0.1163
	INFER _{ONCE}	0.0508	0.0704	0.0072	0.0086	0.0621	0.0876	0.0944	0.1249

- ❑ Vanilla performs well on small datasets
 - ONCE performs best on large datasets
- ❑ Model-agnostic
 - Applicable to various backbones beyond graph-based models

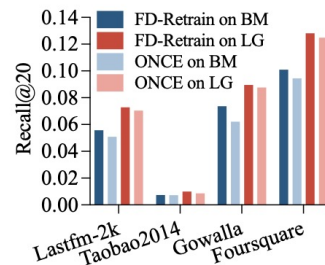
INFER_ONCE vs Full-data Retraining

❖ Compared with full-data retraining

- ❑ CTR: comparable to, or slightly better than, FD-Retrain
 - INFER_ONCE retains user preferences and predicts clicks effectively
- ❑ Top-N: still behind FD-Retrain
 - Ranking requires accurately modeling **relative item order**

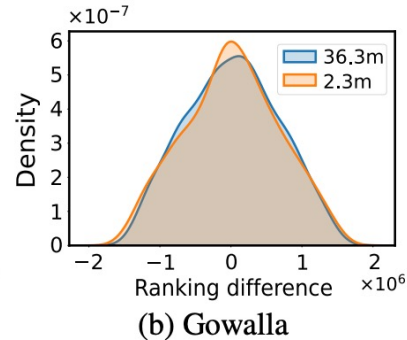
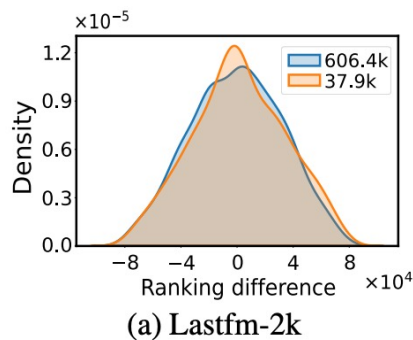
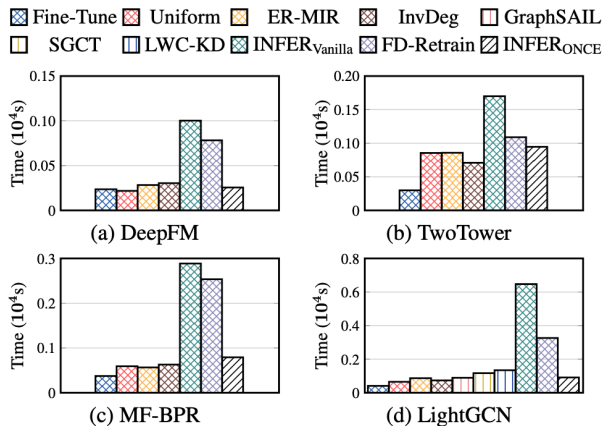


(a) CTR Prediction



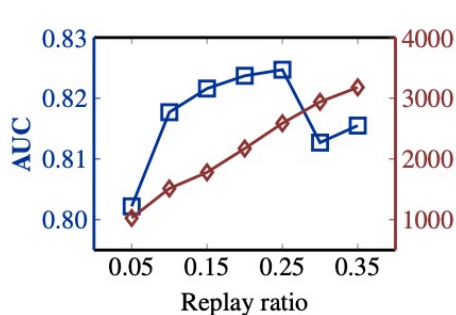
(b) Top-N Recommendation

Empirical Study on INFER_ONCE

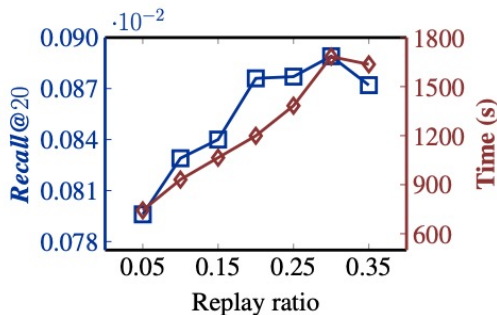


- Measure the **runtime cost** of all incremental models on the largest dataset (Foursquare)
 - Achieves **similar or better performance** than recent incremental recommenders
- Does the vanilla approximation remain accurate across different backbone sizes?
 - Compute influence scores and **rank samples** accordingly
 - Compare the **ranking differences** between the two implementations

Parameter Study on Reservoir Size



(a) TwoTower

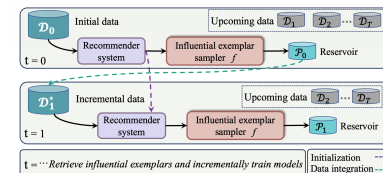
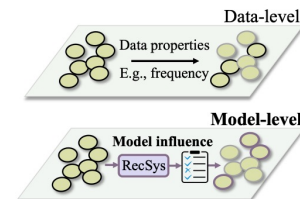


(b) LightGCN

- replay ratio : adjust $K/|D|$ to vary the reservoir size
 - K : number of samples stored in the buffer
 - $|D|$: size of the current training data
- Larger ratios increase training cost and may hurt performance
 - Over-replaying past data can introduce **bias** and reduce adaptation to new data

Conclusion

- ❑ When selecting samples to help the model retain knowledge, choosing them based only on data properties may yield samples that are not actually beneficial from the model's perspective.
- ❑ Therefore, the paper aims to select samples with the largest “influence” given the model's current training state.
This influence is determined by measuring how the **losses of other samples in the reservoir** would change when a particular sample is **upweighted** during training.
- ❑ In addition, the paper proposes an approximation method to improve the efficiency of influence computation
- ❑ Experiments show overall performance improvements, the efficiency and effectiveness of the approximation, and **model-agnostic** characteristics.





Personalized Negative Reservoir for Incremental Learning in Recommender Systems

Antonios Valkanias
McGill University,
Mila, ILLS

Yuening Wang
Huawei Noah's Ark Lab

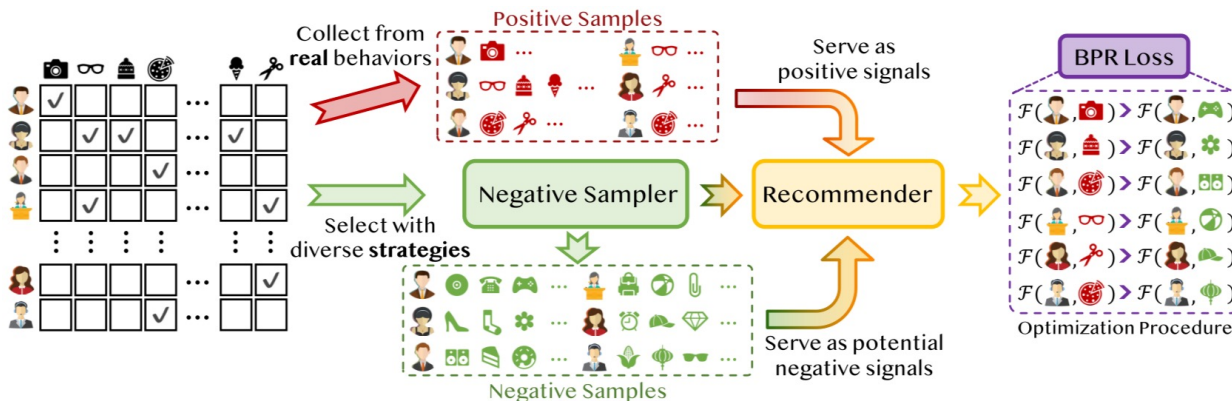
Yingxue Zhang
Huawei Noah's Ark Lab

Mark Coates
McGill University,
Mila, ILLS

Negative Sampling

❖ Negative sampling in recommender systems

- ❑ Select items that the user has not interacted with as negative samples



❖ BPR Loss

- ❑ Encourage the **positive item score** to be higher than the **negative item score**
 - The score reflects how well the item **matches the user**

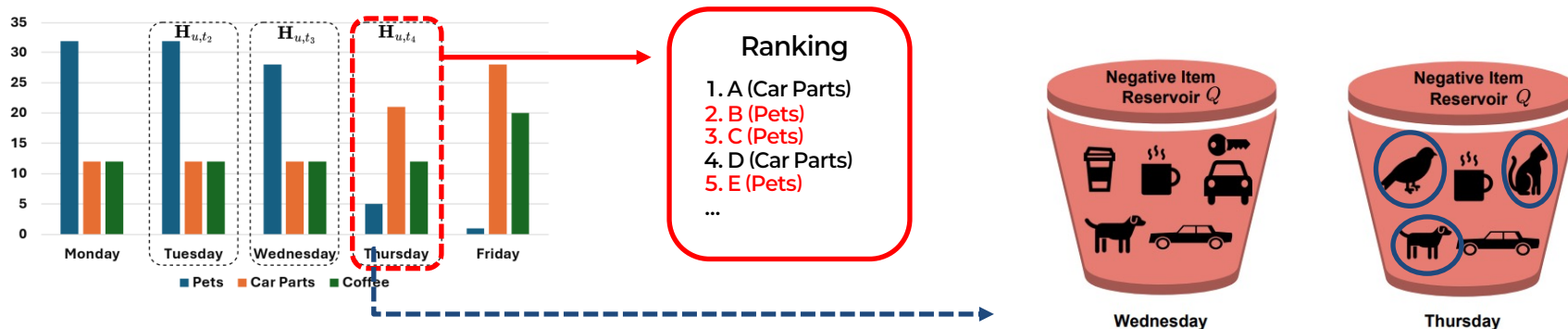
GraphSANE (TMLR' 25)

❖ Focus on Plasticity in Incremental Learning

- Quickly capture and adapt to changing user preferences

❖ Negative sampling tailored for incremental learning

- Sample more negatives from shifted interests
 - We need stronger negative signals to push down items the user is no longer interested in



Which samples help learning?

❖ INFER

- ❑ To preserve the user's long-term preferences learned from past interactions
- ❑ Data sampling for **stability**

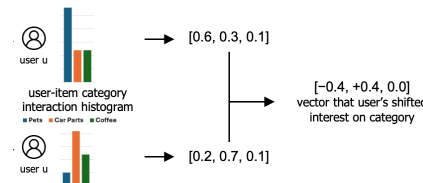
❖ GraphSANE

- ❑ To strongly reflect the user's short-term interests at the current time
- ❑ (Negative) data sampling for **plasticity**

Two hypothesis on Negative Sample

❖ Negative importance should follow interest shift

- Item categories that the user is **less interested in now** compared to the past
 - Sample negative items from these categories to reflect the **shifted interests**
- Estimate the shift using the user's **category-wise interaction proportions** over time

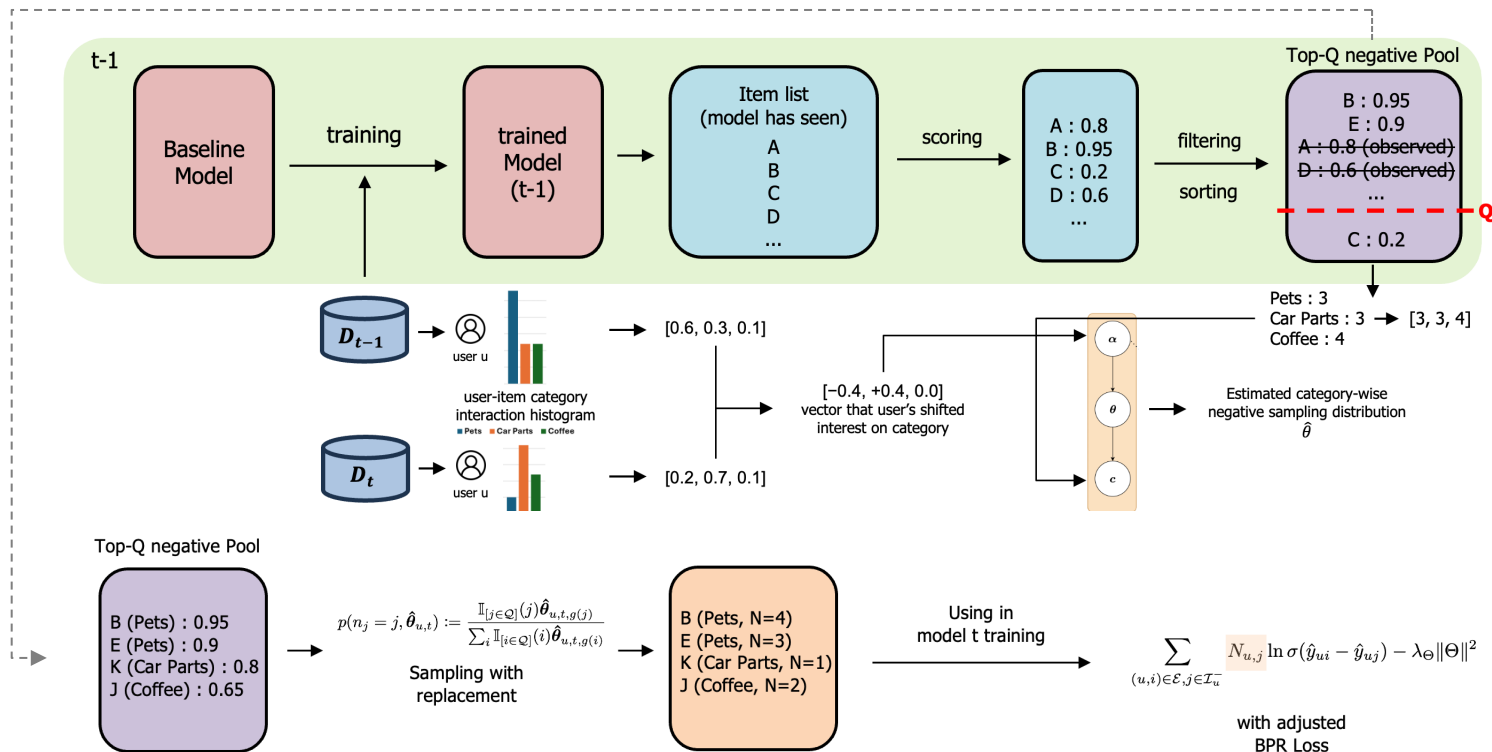


$$\text{shift}(u, t-1, t) = \frac{\mathbf{H}_{u,t}}{|\mathbf{H}_{u,t}|} - \frac{\mathbf{H}_{u,t-1}}{|\mathbf{H}_{u,t-1}|}$$

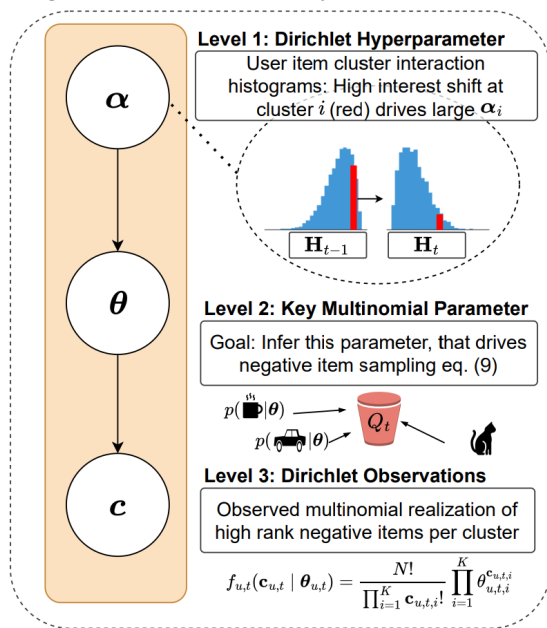
❖ Negatives should still be hard

- Select items that the model ranked highly at time $t-1$, but the user **did not choose**
- Build a negative pool with the **top-Q** such items

Overall View



Estimate the sampling probabilities



❖ Dirichlet Hyperparameter

- Interaction histograms are compared across time to quantify interest shifts

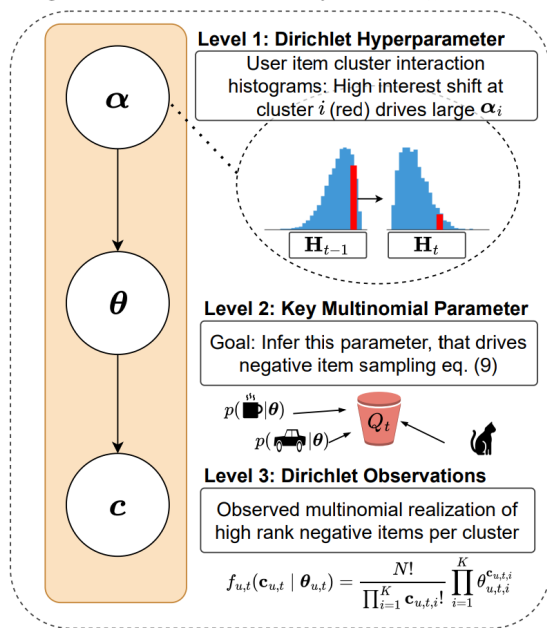
$$\text{LEVEL 1: } \alpha_{u,t} = \lambda Q \text{softmax} \left(- \left[\frac{\mathbf{H}_{u,t}}{|\mathbf{H}_{u,t}|} - \frac{\mathbf{H}_{u,t-1}}{|\mathbf{H}_{u,t-1}|} \right] \right)$$

❖ Key Multinomial Parameter

- The model samples θ from a Dirichlet distribution, category-level probabilities that govern negative item sampling

$$\text{LEVEL 2: } p(\boldsymbol{\theta}_{u,t}) = \text{Dir}(\boldsymbol{\alpha}_{u,t}) = \frac{\Gamma(\alpha_{u,t,0})}{\Gamma(\alpha_{u,t,1}) \dots \Gamma(\alpha_{u,t,K})} \prod_{k=1}^K \theta_{u,t,k}^{\alpha_{u,t,k}-1}$$

Estimate the sampling probabilities (Cont.)



❖ Dirichlet Observations

- ❑ Negative items sampled per category are observed as a multinomial realization

$$\text{LEVEL 3: } f_{u,t}(\mathbf{c}_{u,t} | \theta_{u,t}) = \frac{N!}{\prod_{i=1}^K \mathbf{c}_{u,t,i}!} \prod_{i=1}^K \theta_{u,t,i}^{\mathbf{c}_{u,t,i}}$$

❖ Estimate $\theta_{u,t}$

$$\begin{aligned} p(\theta_{u,t} | \mathbf{c}_{u,t}, \alpha_{u,t}) &\propto p(\theta_{u,t}) f_{u,t}(\mathbf{c}_{u,t} | \theta_{u,t}) = \frac{\Gamma(\alpha_0)}{\Gamma(\alpha_1) \dots \Gamma(\alpha_K)} \prod_{k=1}^K \theta_k^{\alpha_k-1} \frac{N!}{\prod_{i=1}^K \mathbf{c}_i!} \prod_{i=1}^K \theta_i^{\mathbf{c}_i} = \text{Dir}(\alpha_{u,t} + \mathbf{c}_{u,t}) \\ &= \text{Dir} \left(\lambda Q_{\text{softmax}} \left(\frac{\mathbf{H}_{u,t}}{|\mathbf{H}_{u,t}|} - \frac{\mathbf{H}_{u,t-1}}{|\mathbf{H}_{u,t-1}|} \right) + \mathbf{c}_{u,t} \right) \end{aligned} \quad (9)$$

$$\hat{\theta}_{u,t,i} = \mathbb{E}[\theta_{u,t,i} | \mathbf{c}_{u,t,i}] = \frac{\lambda Q_{\text{softmax}} \left(\frac{\mathbf{H}_{u,t,i}}{|\mathbf{H}_{u,t,i}|} - \frac{\mathbf{H}_{u,t-1,i}}{|\mathbf{H}_{u,t-1,i}|} \right) + \mathbf{c}_{u,t,i}}{\sum_i \lambda Q_{\text{softmax}} \left(\frac{\mathbf{H}_{u,t,i}}{|\mathbf{H}_{u,t,i}|} - \frac{\mathbf{H}_{u,t-1,i}}{|\mathbf{H}_{u,t-1,i}|} \right) + \mathbf{c}_{u,t,i}}$$

Key Idea: Sampling Frequency = Training Weight

- ❖ Probability of sampling negative item j for user u at time t

$$p(n_j = j, \hat{\theta}_{u,t}) := \frac{\mathbb{I}_{[j \in \mathcal{Q}]}(j) \hat{\theta}_{u,t,g(j)}}{\sum_i \mathbb{I}_{[i \in \mathcal{Q}]}(i) \hat{\theta}_{u,t,g(i)}}$$

- ❑ (Indicator of whether item j belongs to the top- Q negative set $\{0, 1\}$) $\times$ the estimated probability of j 's category
- ❑ Determined by a probability normalized over all top- Q negative items

- ❖ Negative sampling with replacement

$$\sum_{(u,i) \in \mathcal{E}, j \in \mathcal{I}_u^-} N_{u,j} \ln \sigma(\hat{y}_{ui} - \hat{y}_{uj}) - \lambda_{\Theta} \|\Theta\|^2$$

- ❑ Sample (possibly the same) negative items **multiple times** from the top- Q pool
 - Each item has a **sampling count** (number of times it is drawn)
 - This count **weights** the interaction loss for that item in the BPR loss

Overall Framework

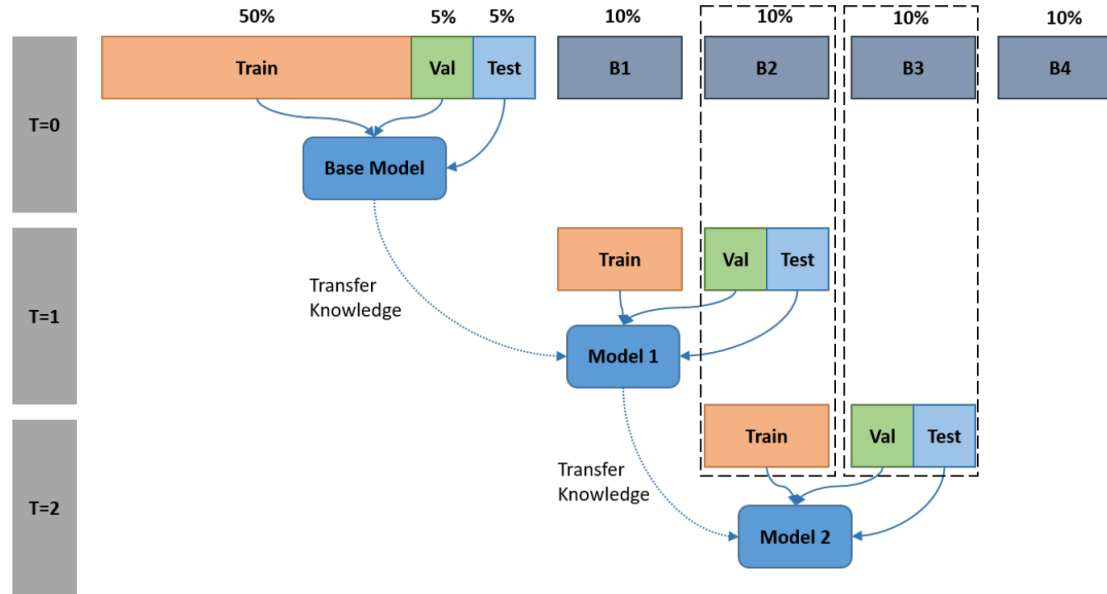
- ❖ End-to-end training with any GNN backbone and any incremental learning framework
 - Orthogonally applicable (plug-and-play)

- ❖ Training becomes unstable when using only hard negatives
 - Mix with random negatives sampled uniformly
 - Improves diversity and training stability

$$\mathcal{L}_{\text{TOTAL}} = \underbrace{\mathcal{L}_{\text{BASE}}}_{\mathcal{L}_{\text{TRIPLET}}, \mathcal{L}_{\text{KD}}} + \underbrace{\mathcal{L}_{\text{SANE}}}_{\text{Neg. Reservoir}}$$

Experiment Setup

❖ Evaluation Benchmarks



Baselines

❖ Incremental learning baselines

- ❑ Fine-Tune
- ❑ LSP_s
 - A knowledge distillation (KD) method tailored for GCNs
- ❑ Uniform
 - Random replay of past data
- ❑ Inv_degree
 - A SOTA replay strategy that stores past interactions based on **inverse degree**
- ❑ GraphSAIL, SGCT, LWC-KD
 - Recent KD-based incremental learning methods
- ❑ Backbone recommender: MGCCF
 - All incremental methods are implemented on the **same backbone**

Baselines (Cont.)

❖ Negative sampler baselines

- ❑ WARP, PNS, PinSage sampler, NS Cache, ESNS, DeMix

Comparison to SOTA Incremental Learning

	Gowalla-20		Taobao2014		Taobao2015		Yelp		Netflix		MovieLens10M	
Methods	Average Rec@20	$\Delta\%$	Average Rec@20	$\Delta\%$	Average Rec@20	$\Delta\%$	Average Rec@20	$\Delta\%$	Average Rec@20	$\Delta\%$	Average Rec@20	$\Delta\%$
Fine Tune	0.1705	0.0	0.0153	0.0	0.0950	0.0	0.0661	0.0	0.1036	0.0	0.0918	0.0
LSP_s	0.1783	4.6	0.0152	-0.7	0.0968	1.9	0.0676	2.3	0.1128	8.7	0.0925	0.8
Uniform	0.1728	1.3	0.0157	2.6	0.0982	3.4	0.0654	-1.1	0.0957	-7.6	0.0885	-3.6
Inv_degree	0.1738	1.9	0.0175	14.6	0.0989	4.2	0.0677	2.4	0.0957	-7.6	0.0915	-0.3
GraphSAIL	0.1849	8.4	0.0155	1.3	0.0963	1.4	0.0639	-3.3	0.1051	1.4	0.0890	-3.1
GraphSAIL-SANE*	0.1925	12.9	0.0171	11.8	0.1107	16.5	0.0857	29.7	0.1086	4.8	0.0921	0.3
SGCT	0.1870	9.7	0.0160	1.7	0.0999	5.2	0.0668	1.06	0.1135	9.6	0.0919	0.1
SGCT-SANE*	0.1897	11.3	0.0178	16.3	0.1128	18.7	0.0862	30.4	0.1155	11.5	0.0946	3.1
LWC-KD	0.1977	15.9	0.0176	15.3	0.1030	8.4	0.0679	2.7	0.1142	10.2	0.0928	1.1
LWC-KD-SANE*	0.1881	10.3	0.0188	22.9	0.1114	17.2	0.0898	35.9	0.1182	14.1	0.0964	5.0

- Compare the base models with/without the **negative reservoir**
 - Achieves the best performance on almost all datasets **except the smallest one (Gowalla-20)**
 - ~5,000 items, ~50 interactions per user on average
 - Sampling **10 negatives per interaction** $\Rightarrow$ about **10% of all items** are used as negatives per epoch
 - With only **10-20 epochs**, it is likely that **almost all negative items** get sampled at least once

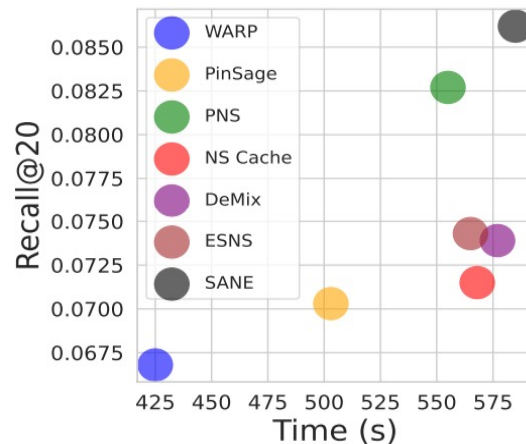
Comparison to SOTA Incremental Learning

	Recall@20 Rank (↓)	NDCG@20 Rank (↓)	Precision@20 Rank (↓)	MAP@20 Rank (↓)
GraphSail	6.00	4.75	5.00	5.00
GraphSail+SANE	3.00	3.75	3.25	3.50
SGCT	4.75	4.75	4.50	4.50
SGCT+SANE	2.25	2.75	3.25	2.00
LWCKD	3.25	3.25	3.75	3.25
LWCKD+SANE	1.50	2.00	1.50	2.50

- Shows clear improvements not only in **Recall** but also across **other metrics**

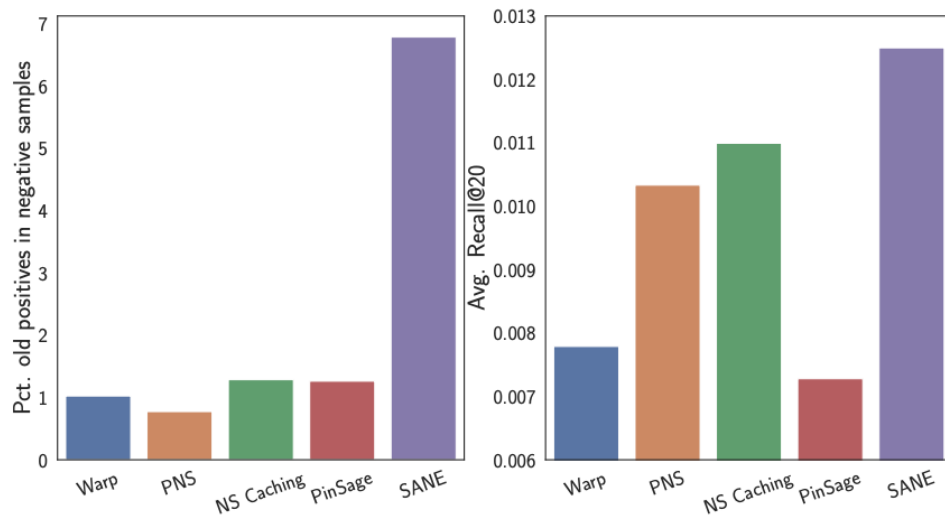
Comparison to Generic Neg. Samplers

Dataset	Methods	Inc 1	Inc 2	Inc 3	Avg.
Yelp	SGCT+[Warp]	0.0740	0.0656	0.0608	0.0668
	SGCT+[PinSage]	0.0794	0.0663	0.0651	0.0703
	SGCT+[PNS]	0.0933	0.0798	0.0748	0.0827
	SGCT+[NS Cache]	0.0794	0.0681	0.0670	0.0715
	SGCT+[DeMix]	0.0783	0.0740	0.0694	0.0739
	SGCT+[ESNS]	0.0786	0.0754	0.0690	0.0743
	SGCT+[SANE] (ours)	0.0966	0.0877	0.0744	0.0862
Taobao14	SGCT+[Warp]	0.0240	0.0092	0.0148	0.0160
	SGCT+[PinSage]	0.0241	0.0099	0.0151	0.0164
	SGCT+[PNS]	0.0220	0.0114	0.0113	0.0149
	SGCT+[NS Cache]	0.0237	0.0124	0.0121	0.0165
	SGCT+[DeMix]	0.0196	0.0097	0.0113	0.0135
	SGCT+[ESNS]	0.0218	0.0113	0.0128	0.0153
	SGCT+[SANE] (ours)	0.0224	0.0136	0.0173	0.0178



- ❑ Comparison of negative samplers under the incremental learning framework SGCT
 - Evaluate **Recall@20** for each **incremental data block**
 - Results when using **hard negatives only** (not random + hard)
- ❑ Achieves the **best performance** among methods with the **same time budget**

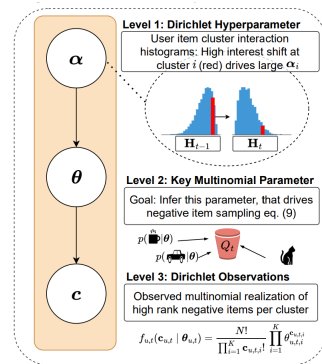
Case Study: Dynamic Users



- 15% of Users with high interest shift
 - Samples **7× more negatives** from the **old positive categories** than other negative samplers
 - Recall@20 improves by ~20%
 - For the remaining **85%** of users with **smaller interest shifts**, Recall@20 improves by ~15%

Conclusion

- In incremental learning, we can capture changes in users' interests, and leverage them to provide **hard negatives** that better reflect these shifts.
- Treat the user's **interest shift** as a **prior**, use the model's **top-Q negative samples** to obtain the **category distribution** as evidence, and estimate **category-wise negative sampling probabilities** for sampling.
- Experiments show that method improves baseline models in a **plug-and-play** manner, and evaluate both **effectiveness and efficiency** against other negative sampling strategies. Case study shows that it works particularly well for **dynamic users**.



Appendix. NDCG

Rank	News	Click 여부
1	A	0
2	B	0
3	C	1 (정답)
4	D	0
5	E	1 (정답)

$$DCG = \frac{1}{\log_2(\text{rank} + 1)}$$

- DCG@10 =

$$\frac{1}{\log_2(3 + 1)} + \frac{1}{\log_2(5 + 1)} = 0.5 + 0.386 = 0.886$$

- IDCG@10 (정답이 1위, 2위였다고 가정)

$$1 + \frac{1}{\log_2(3)} = 1 + 0.6309 = 1.6309$$

- NDCG@10 =

$$0.886/1.6309 = 0.543$$

Appendix. SANE_OPT

$$\begin{aligned}
 \text{SANE}_{\text{OPT}} &:= \ln p(\Theta \mid \succ_u) \propto \ln (p(\succ_u \mid \Theta) p(\Theta)) \\
 &= \ln \left(\prod_{(u,i) \in \mathcal{E}, j \in \mathcal{I}_u^-} \sigma(\hat{y}_{ui} - \hat{y}_{uj})^{N_{u,j}} p(\Theta) \right) \\
 &= \sum_{(u,i) \in \mathcal{E}, j \in \mathcal{I}_u^-} N_{u,j} \ln \sigma(\hat{y}_{ui} - \hat{y}_{uj}) - \lambda_{\Theta} \|\Theta\|^2,
 \end{aligned}$$